

المحاضرة 8

الدوال (التوايع) 2: functions
مفاهيم متقدمة في التوايع ضمن c++
د. سهيل الحمود
د. أسامه ناصر

2025-2026

المتحولات الساكنة

```
#include<iostream>
using namespace std;
int g=0;
int fun1(int p1,int p2){
    static int tmp=0;
    tmp+=p1+p2;g+=p1+p2;
    return tmp;
}
int main(){
    cout<<fun1(1,3)<<"\n"
    <<g<<endl;
    cout<<fun1(1,3)<<"\n"
    <<g<<endl;
    return 0;
}
```

- بفرض لدينا الكود التالي:
- لدينا متحول اسمه g تم تعريفه ضمن الملف وليس ضمن أي مجال رؤية
- اي أن هذا المتحول مرئي على كامل الملف
- المتحول tmp هو متحول ساكن ضمن التابع fun1
- الخرج

```
4
4
8
8
```

المتحولات الساكنة

- هو نوع من المتحولات يعرف بالكلمة المفتاحية static ضمن مجال الرؤية المعرف ضمنه
- يعامل مثل التعامل مع متحولات عامة، إنما ضمن مجال الرؤية حصراً
- تتم تهيئة قيمة المتحول عند أول استدعاء للتابع، وتنتقل القيمة مع كل التعديلات التي تمر عليها من استدعاء لآخر

المتحولات الساكنة

تنفيذ البرنامج خطوة بخطوة

- استدعاء التابع fun1 للمرة الأولى
- تهيئة المتحول الساكن tmp
- تنفيذ تغيير القيم لكل من tmp و g
- إعادة القيمة
- طباعة النتيجة
- استدعاء التابع fun1 للمرة الثانية
- لا نهى المتحول tmp وإنما نتقل مباشرة لتغيير القيم
- إعادة القيمة
- طباعة النتيجة
- إنهاء البرنامج

```
#include<iostream>
using namespace std;
int g=0;
int fun1(int p1,int p2){
    static int tmp=0;
    tmp+=p1+p2;g+=p1+p2;
    return tmp;
}
int main(){
    cout<<fun1(1,3)<<"\n"
    <<g<<endl;
    cout<<fun1(1,3)<<"\n"
    <<g<<endl;
    return 0;
}
```

توسعة في مجالات الرؤية

- مجال الرؤية هو المجال الذي يمكن فيه استخدام المتحولات
- يعرف عادة بين قوسين {}
- قواعد عامة ضمن مجالات الرؤية
- التعريف ضمن المجال ويكون فيه المتحول غير مرئي خارجه
- يمكن لمجالات الرؤية أن تتداخل وبالتالي متحولات الأب مرئية ضمن متحولات الأبناء
- التابع مجال رؤية مستقل بحد ذاته، أي إذا استدعينا تابع ضمن مجال لا يمكنه رؤية المتحولات للمجال الذي استدعاه
- لا يمكن رؤية متحولات مجال الرؤية الابن ضمن مجال الرؤية الأب

توسعة في مجالات الرؤية

```
#include<iostream>
using namespace std;
int main(){
    for(int i=0;i<10;i++){
        // ✓ correct code
        cout<<i%2<<endl;
    }
    // ✗ bad code, compiler fails
    cout<<i;
}
```

- قواعد عامة ضمن مجالات الرؤية
- التعريف ضمن المجال ويكون فيه المتحول غير مرئي خارجه
- لا يمكن رؤية متحولات مجال الرؤية الابن ضمن مجال الرؤية الأب
- في هذا المثال المتحول i معرف ضمن كتلة حلقة for وبالتالي غير مرئي خارجها

توسعة في مجالات الرؤية

```
#include<iostream>
using namespace std;
int main(){
    int i;
    for(i=0;i<10;i++){
        // ✓ correct code
        cout<<i%2<<endl;
    }
    // ✓ correct code
    cout<<i;
}
```

- قواعد عامة ضمن مجالات الرؤية
- يمكن لمجالات الرؤية أن تتداخل وبالتالي متحولات الأب مرئية ضمن متحولات الأبناء
- في هذا المثال المتحول i معرف خارج كتلة حلقة for ضمن كتلة main وبالتالي مرئي ضمن مجال رؤية main و for (الأب والابن)

توسعة في مجالات الرؤية

```
int sum(int a,int b){
    int x=a+b;
    //X bad code, compiler fails
    c=1;
    return x
}
int main(){
    cout<<i;
    int c=10;
    cout<<sum(1,4);
    return 0;
}
```

- قواعد عامة ضمن مجالات الرؤية
- التابع مجال رؤية مستقل بحد ذاته، أي إذا استدعينا تابع ضمن مجال لا يمكنه رؤية المتحولات للمجال الذي استدعاه
- السبب مرتبط بشكل أساسي بكيفية التعامل مع التوابع
- لا يوجد شرط على استدعاء التابع سوى أن يكون ضمن تابع آخر (على الأقل ضمن الـ main)
- وبالتالي لا يمكننا تحديد مجال الرؤية الفعلي الذي استدعى هذا التابع وبناء على ذلك لا يمكننا تحديد المتحولات التي يمكن للتابع رؤيتها ضمن مجال الأب من عدمها
- كما يمكن استدعاء التابع من ملفات أخرى

توسعة في مجالات الرؤية

```
int sum(int a,int b){
    int x=a+b;
    //X bad code, compiler fails
    c=1;
    return x
}
int main(){
    cout<<i;
    int c=10;
    cout<<sum(1,4);
    return 0;
}
```

- قواعد عامة ضمن مجالات الرؤية
- التابع مجال رؤية مستقل بحد ذاته، أي إذا استدعينا تابع ضمن مجال لا يمكنه رؤية المتحولات للمجال الذي استدعاه
- كما يعد هذا الأمر ميزة أمنية مهمة
- لا يمكنني تعديل سلوك التابع ولا تغيير العمليات داخله،
جل استطاعتنا تغيير القيم الممررة له
- بالتالي نضمن استقرار الأداء، دخل محدد => خرج محدد

توابع شهيرة بنت بهيرة

توابع رياضية

- تتواجد هذه التوابع ضمن المكتبة `cmath`
- `sqrt`: لحساب الجذر التربيعي
- `pow`: لحساب قوة (وسيطين العدد والقوة)
- `ceil`: السقف (أصغر عدد صحيح أكبر أو يساوي العدد المعطى،
مثلاً سقف 20.1 هو 21)
- `floor`: القاع (أكبر عدد أصغر أو يساوي العدد المعطى، مثلاً
قاع 20.9 هو 20)
- `round`: التقريب وفقاً لقواعد التقريب الرياضي
- `abs`: القيمة المطلقة
- `exp`: التابع الأسّي e^x
- `log`: اللوغاريتم الطبيعي للعدد
- `log10`: اللوغاريتم العشري للعدد
- `sin`: جيب الزاوية
- `cos`: جيب التمام للزاوية
- `tan`: ظل الزاوية

توابع شهيرة بنت بهيرة

ضبط شكل الخرج

```
#include<iostream>
#include<iomanip>
using namespace std;
int main(){
double pi=22.0/7,e=2.79346752;
cout<<"Pi"<<setw(15)<<pi<<endl;
cout<<"e"<<setw(7)
    <<setprecision(5)
    <<e<<endl;
return 0;
}
```

```
Pi          3.14286
e 2.7935
```

- setw
- ضبط فراغات في الخرج
- setprecision
- تغيير عدد الأعداد بعد الفاصلة أثناء طباعة الأعداد الحقيقية
- خرج المثال

التوليد العشوائي

- توليد القيم العشوائية مهم جدًا في علوم الحاسب
- خصوصًا في مجال أمن المعلومات لتوليد مفاتيح التشفير
- تعتمد شركة Cloudflare على مجموعة مصابيح ذات حركة معقدة جدًا لتوليد المفاتيح

التوليد العشوائي

- التابع المسؤول عن التوليد العشوائي هو rand الموجود ضمن المكتبة `stdlib`
- لا يتطلب أي وسيط، خرج عدد بين 0 و `RAND_MAX`
- قيمة `RAND_MAX` مختلفة من نسخة لأخرى لكن على الأقل تكون 32767
- كيف يمكن أن تحكم بقيم الخرج لتصبح ضمن مجال محدد؟
- الحل بالاعتماد على `%` و `-`
- نريد توليد قيمة بين 0 و 6
 - `rand()%7`
- ناتج باقي قسمة أي عدد على 7 يقع ضمن المجال `[0,6]`

التوليد العشوائي

- مثال
- خرج البرنامج
- أول تشغيل

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int main(){
    cout<<rand()<<endl;
    cout<<rand()%7<<endl;
    cout<<rand()%11-5<<endl;
    return 0;
}
```

```
1804289383
4
1
```

- ثاني تشغيل

```
1804289383
4
1
```

التوليد العشوائي

- كيف برنامج عشوائي يولد خرجين متطابقين ؟
- ألا يفترض ان يولد أعدادًا عشوائية عند كل تشغيل
- عم تتخوت علينا دكتور؟
- صراحةً وفعليًا لا
- البرنامج يعمل بشكل نظامي ولكن آلية العمل غير واضحة
- في الطبيعة لا يوجد عشوائية كل شيء يتبع لقاعدة أو نمط مهما كان معقدًا هذا النمط
- حتى في الحاسب، تابع rand يتبع معادلة رياضية معقدة لتوليد قيم شبه عشوائية pseudo-random
- كأى معادلة تحتاج لنقطة بداية وبالتالي في حال معرفة البداية يمكن معرفة النهاية
- أي مهما استدعينا، إذا قيمة البداية ستبقى دائمًا النهاية نفسها
- بناءً على ذلك يجب أن نغير نقطة البداية عند بدء البرنامج لتتغير النتيجة عند كل استدعاء

التوليد العشوائي

- خرج البرنامج
- أول تشغيل

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int main(){
    srand(time(0));
    cout<<rand()<<endl;
    cout<<rand()%7<<endl;
    cout<<rand()%11-5<<endl;
    return 0;
}
```

```
2143678813
4
3
```

- ثاني تشغيل

```
1843619091
0
-1
```

التوليد العشوائي

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int main(){
    srand(time(0));
    cout<<rand()<<endl;
    cout<<rand()%7<<endl;
    cout<<rand()%11-5<<endl;
    return 0;
}
```

- نلاحظ الاختلاف في السطر المحدد
- يستمح لنا التابع `srand` بتغيير نقطة البداية لتابع `rand`
- التابع `time` يعيد لنا القيمة الحالية لوقت النظام مخزنة كثنائي ابتداءً من 00:00 تاريخ 1 كانون الثاني 1970 (Unix Time)
- هذه القيمة تتغير بشكل مستمر نتيجة عد الثواني بشكل مستمر، بناءً على ذلك عند كل تشغيل للبرنامج نتحصل على نقطة بداية جديدة لذلك تتغير القيمة عند كل تشغيل

مثال كبير قوي

لنعمل على كتابة برنامج يحاكي رمي النرد (الزهر) 100000 مرة

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int dice[]={0,0,0,0,0,0};
void throwDice(){
    int diceValue=rand()%6;
    dice[diceValue]++;
}
```

```
int main(){
    srand(time(0));
    for(int i=0;i<100000;i++)
        throwDice();
    cout<<"The dice\n";
    for(int i=0;i<6;i++)
        cout<<"\t"<<i+1<<" "<<dice[i]
<<endl;
    return 0;
}
```

مثال كبير قوي

لنعمل على كتابة برنامج يحاكي رمي النرد (الزهر) 100000 مرة

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int dice[]={0,0,0,0,0,0};
void throwDice(){
    int diceValue=rand()%6;
    dice[diceValue]++;
}
```

- تعريف مصفوفة (تفاصيل أكثر المحاضرة القادمة)
- تعريف تابع يقوم برمي الزهر
- نأخذ قيمة عشوائية بين 0 و 5
- نقوم بزيادة القيمة المخزنة في المصفوفة موافقة للرقم المولد (وجه النرد) بمقدار 1

مثال كبير قوي

لنعمل على كتابة برنامج يحاكي رمي النرد (الزهر) 100000 مرة

```
int main(){
    srand(time(0));
    for(int i=0;i<100000;i++)
        throwDice();
    cout<<"The dice\n";
    for(int i=0;i<6;i++)
        cout<<"\t"<<i+1<<" "<<dice[i]
<<endl;
    return 0;
}
```

- تعيين قيمة بداية لتابع rand
- حلقة من 100000 تكرار
- تنفيذ استدعاء التابع throwDice
- حلقة للمرور على أوجه النرد
- طباعة قيمة التكرار لكل وجه

مثال كبير قوي

خرج البرنامج

```
The dice
1 16777
2 16668
3 16626
4 16710
5 16516
6 16703
```